

Final Report

Healthcare Data Platform using Medallion Architecture

Solution Design & Implementation Report

Prepared By: Kushagra Sanghi

Internship: Celebal Technologies – Data Engineering Internship

Project Duration: Week 2

Version: 1.0

Document Structure

1. Executive Summary
2. Understanding the Problem Definition Document (PDD)
3. Solution Design & Architecture
4. Security, Privacy & Compliance Engineering
5. Project Architecture
6. Notebook-wise Implementation
7. Engineering Decisions & Design Rationale
8. Results, Challenges & Future Enhancements
9. Conclusion
10. References & Appendix

Chapter 1 — Executive Summary

1.1 Introduction

Healthcare organizations generate large volumes of operational data every day through patient registrations, appointments, consultations, treatments, billing transactions, and administrative processes. While this data is essential for clinical operations, it is often distributed across multiple systems and stored in raw formats that are unsuitable for analytical reporting or strategic decision-making.

The objective of this project was to design and implement an end-to-end healthcare data platform using the Medallion Architecture on Databricks. The platform ingests raw healthcare datasets, progressively transforms them into trusted analytical assets, and generates business-ready insights while maintaining data quality, historical consistency, and operational transparency.

The implementation follows the architectural guidance provided in the Project Definition Document (PDD) while incorporating additional engineering improvements to enhance maintainability, scalability, security, and compliance with healthcare data management best practices.

1.2 Business Problem

Healthcare institutions rely on accurate and timely information to support operational planning, patient care, financial management, and regulatory reporting. However, raw transactional data presents several challenges:

- Data originates from multiple independent sources with varying quality and structure.
- Missing values, duplicate records, and inconsistent formats reduce data reliability.
- Historical changes to business entities must be preserved without overwriting previous information.
- Healthcare data contains sensitive information that requires secure handling and privacy protection.
- Manual reporting processes increase effort and reduce decision-making efficiency.
- Limited pipeline monitoring makes troubleshooting and operational maintenance difficult.

Without a structured data engineering solution, organizations face difficulties in producing reliable analytics, maintaining regulatory compliance, and supporting business intelligence initiatives.

1.3 Project Objectives

The primary objective of this project is to implement a scalable healthcare data platform capable of transforming raw operational data into trusted analytical datasets through a structured Medallion Architecture.

The project specifically aims to:

- Design a layered data engineering pipeline using Bronze, Silver, and Gold data layers.
 - Ingest multiple healthcare datasets into a centralized data lake.
 - Perform data validation, cleansing, and business transformations.
 - Implement Slowly Changing Dimension (SCD Type 2) to preserve historical data changes.
 - Generate analytical KPIs for patient, appointment, treatment, and billing operations.
 - Build a comprehensive audit logging framework for pipeline monitoring.
 - Apply healthcare-oriented security measures, including business hashing of identifiers.
 - Maintain a modular notebook structure for improved maintainability and extensibility.
-

1.4 Scope of Implementation

The implemented solution includes the complete lifecycle of healthcare data processing, beginning with raw CSV ingestion and ending with business-ready analytical datasets.

The implementation consists of:

- Environment configuration and metadata management
- Bronze layer ingestion of raw healthcare datasets
- Silver layer validation and business transformations
- Historical tracking using SCD Type 2
- Gold layer KPI generation
- Pipeline execution monitoring through audit logging
- Secure handling of sensitive business identifiers using hashing techniques

The project was implemented entirely using Databricks, Apache Spark (PySpark), and Delta Lake.

1.5 Key Deliverables

The final solution delivers:

- Metadata-driven healthcare data pipeline
 - Eight modular Databricks notebooks (00–07)
 - Bronze, Silver, and Gold Delta tables
 - Healthcare KPI reporting tables
 - Audit logging framework
 - Historical data tracking through SCD Type 2
 - Business hashing implementation for sensitive identifiers
 - GitHub repository with complete documentation
 - Technical implementation report
-

1.6 Technologies Used

Component	Technology
Platform	Databricks
Processing Engine	Apache Spark (PySpark)
Storage	Delta Lake
Programming Language	Python
Data Source	CSV Files
Architecture	Medallion Architecture
Historical Tracking	SCD Type 2
Security	SHA-256 Business Hashing
Monitoring	Audit Logging
Version Control	Git & GitHub

1.7 Project Outcome

The completed healthcare data platform successfully transforms raw healthcare datasets into reliable analytical assets through a scalable and modular Medallion Architecture. The solution

satisfies the primary functional requirements defined in the PDD while introducing several engineering enhancements, including improved audit logging, standardized metadata management, business hashing for sensitive identifiers, and healthcare-oriented design considerations aligned with HIPAA principles.

The resulting platform establishes a robust foundation for future reporting and visualization tools such as Power BI while remaining extensible for incremental data processing, workflow automation, and enterprise-scale deployment.

Chapter 2 — Understanding the Problem Definition Document (PDD)

2.1 Introduction

The Project Definition Document (PDD) serves as the foundational blueprint for developing the Healthcare Data Platform. It defines the business objectives, functional requirements, architectural expectations, and implementation guidelines for building a scalable data engineering solution using the Medallion Architecture.

Rather than functioning as a coding specification alone, the PDD establishes a structured framework for transforming raw healthcare data into reliable analytical assets while ensuring data quality, maintainability, and operational transparency. It outlines the expected data flow, processing layers, security considerations, and reporting requirements that collectively support business intelligence and decision-making.

This chapter analyzes the PDD in detail, identifies its primary requirements, and explains how those requirements shaped the overall implementation strategy adopted in this project.

2.2 Business Context

Healthcare organizations generate large volumes of operational data across various departments, including patient registration, clinical consultations, appointments, treatment records, and billing operations. These datasets are typically collected through independent operational systems and often differ in structure, quality, and completeness.

Although these systems efficiently support day-to-day healthcare operations, they are generally not optimized for analytical reporting or enterprise-level decision-making. Data inconsistencies, duplicate records, missing values, and fragmented storage make it difficult to derive accurate business insights directly from transactional data.

To address these challenges, organizations require a centralized data platform capable of consolidating, validating, transforming, and organizing healthcare information into trusted analytical datasets. Such a platform should support historical tracking, maintain high data quality, provide operational visibility through monitoring mechanisms, and ensure that sensitive healthcare information is handled securely.

The PDD addresses these business challenges by proposing the implementation of a Medallion Architecture on Databricks, enabling progressive refinement of healthcare data from raw ingestion to business-ready analytics.

2.3 Problem Statement

The primary objective defined by the PDD is to design and implement a scalable healthcare data platform capable of converting raw operational healthcare datasets into trusted analytical assets through a structured, multi-layered data engineering pipeline.

The proposed solution is expected to:

- Ingest multiple healthcare datasets into a centralized storage layer.
- Preserve raw source data without modification.
- Validate and cleanse incoming records to improve data quality.
- Apply business transformations required for downstream analytics.
- Maintain historical versions of changing business entities using Slowly Changing Dimension (SCD Type 2).
- Generate business-ready Key Performance Indicators (KPIs) for operational reporting.
- Implement audit logging to monitor pipeline execution and support troubleshooting.
- Provide a modular and maintainable architecture that can be extended as business requirements evolve.

By implementing these capabilities, the platform enables healthcare organizations to transition from isolated transactional data toward a reliable and scalable analytical ecosystem.

2.4 Objectives Defined in the PDD

The PDD specifies several technical and business objectives that collectively define the scope of the project. These objectives guided the design and implementation of the final solution.

Raw Data Ingestion

The pipeline should ingest multiple healthcare datasets into the Bronze layer while preserving the original records without modification. This layer serves as the system of record for all incoming data.

Data Quality Improvement

The solution should validate incoming datasets, identify invalid records, standardize formats, and improve overall data consistency before analytical processing.

Business Transformation

Healthcare data should be transformed into business-friendly structures that simplify reporting, improve usability, and support downstream analytical workloads.

Historical Data Management

The platform should preserve historical changes to selected business entities through Slowly Changing Dimension (SCD Type 2), allowing organizations to perform trend analysis and historical reporting.

Analytical Reporting

The Gold layer should provide curated datasets containing operational KPIs related to patients, appointments, treatments, and billing activities.

Pipeline Monitoring

The implementation should maintain execution logs that capture processing status, record counts, execution timestamps, and operational details to improve observability and simplify troubleshooting.

Modular Pipeline Design

Each processing stage should be implemented as an independent notebook, improving maintainability, readability, testing, and future extensibility.

2.5 Functional Requirements

Based on the PDD, the platform is expected to provide the following functional capabilities:

Requirement	Description
Data Ingestion	Load healthcare CSV datasets into Bronze tables.
Data Validation	Identify missing, duplicate, and inconsistent records.
Data Transformation	Apply business rules and standardize data formats.
Historical Tracking	Maintain historical versions using SCD Type 2.
KPI Generation	Produce business-ready analytical tables in the Gold layer.
Audit Logging	Record execution details for each pipeline stage.
Metadata Management	Maintain reusable configuration for pipeline execution.

2.6 Non-Functional Requirements

In addition to functional capabilities, the PDD emphasizes several quality attributes that influence the overall system design.

Scalability

The pipeline should be capable of processing increasing data volumes without requiring significant architectural modifications.

Maintainability

Processing logic should be organized into modular notebooks with clearly defined responsibilities, enabling easier debugging, testing, and future enhancements.

Reliability

The platform should ensure consistent execution, robust error handling, and comprehensive audit logging to support operational monitoring.

Performance

Apache Spark should be leveraged for distributed processing, enabling efficient execution of transformations on large healthcare datasets.

Security

Sensitive healthcare identifiers should be protected through appropriate engineering controls, ensuring that confidential information is not unnecessarily exposed during downstream processing.

Extensibility

The architecture should support future integration with visualization tools, workflow orchestration, streaming pipelines, and advanced analytical capabilities without requiring major redesign.

2.7 Expected Project Deliverables

According to the PDD, the project is expected to deliver a complete healthcare data engineering pipeline comprising multiple processing stages.

The primary deliverables include:

- Environment configuration and metadata management.
- Bronze layer for raw data ingestion.
- Silver layer for validation and business transformations.
- Historical data management using SCD Type 2.
- Gold layer containing analytical KPI tables.
- Audit logging framework for execution monitoring.
- Modular Databricks notebooks implementing each processing stage.

- Architecture documentation describing the overall solution design.

In addition to these deliverables, the implementation presented in this report incorporates several engineering enhancements that extend beyond the original PDD, including business hashing for sensitive identifiers, an enhanced audit framework, improved metadata management, and healthcare-oriented design considerations aligned with HIPAA principles. These enhancements are discussed in detail in later chapters.

2.8 Transition to the Solution Design

The PDD establishes a clear vision for building a modern healthcare data platform but intentionally leaves several implementation decisions open to engineering judgment. Throughout the development process, multiple architectural choices were made to improve modularity, maintainability, security, and operational visibility while remaining aligned with the original business objectives.

The following chapter explains the overall solution architecture, design rationale, and technology choices that guided the implementation of the Healthcare Data Platform. It discusses why Databricks, Apache Spark, Delta Lake, and the Medallion Architecture were selected, and how these technologies collectively satisfy both the functional and non-functional requirements defined in the PDD.

Chapter 3 — Solution Design & Architecture

3.1 Introduction

The Healthcare Data Platform was designed to transform raw operational healthcare data into trusted analytical datasets through a structured and scalable data engineering pipeline. While the Project Definition Document (PDD) defined the overall objectives, the implementation required several architectural and engineering decisions to ensure the solution remained maintainable, extensible, secure, and suitable for real-world healthcare environments.

To achieve these objectives, the project adopts the **Medallion Architecture** on **Databricks**, leveraging **Apache Spark (PySpark)** for distributed data processing and **Delta Lake** for reliable transactional storage. The pipeline progressively refines data across multiple processing layers, allowing each stage to focus on a specific responsibility while maintaining clear separation of concerns.

This layered approach improves data quality, simplifies debugging, enables historical tracking, and produces analytics-ready datasets that can be consumed by reporting and business intelligence tools.

3.2 Solution Overview

The implemented solution follows a sequential data engineering workflow beginning with raw healthcare datasets and ending with business-ready analytical tables.

The pipeline processes data through the following stages:

1. Raw Data Sources

- Patient Records
- Doctor Records
- Appointment Records
- Treatment Records
- Billing Records

2. Bronze Layer

- Raw data ingestion
- Schema preservation
- Source traceability

3. Silver Layer

- Data validation
- Data cleansing
- Standardization
- Business transformations
- Business hashing
- Slowly Changing Dimension (SCD Type 2)

4. Gold Layer

- Business KPIs
- Aggregated reporting tables
- Executive analytics

5. Audit Framework

- Pipeline monitoring
- Execution tracking
- Error reporting
- Operational statistics

This layered workflow ensures that data quality improves progressively while preserving the integrity of the original source data.

3.3 Why Medallion Architecture?

The Medallion Architecture was selected because it provides a structured approach to organizing data according to its level of refinement. Instead of performing all transformations within a single processing stage, data is progressively improved as it moves through independent layers.

This architecture offers several advantages for healthcare data engineering:

- Preserves original source records for traceability.
- Isolates validation and cleansing from business transformations.
- Simplifies debugging by separating processing responsibilities.
- Enables historical data management without modifying raw data.
- Produces highly reliable analytical datasets.

- Supports future scalability as new datasets or business requirements are introduced.

By separating ingestion, transformation, and analytics into distinct layers, the architecture reduces pipeline complexity while improving maintainability and operational transparency.

3.4 Bronze Layer Design

The Bronze layer serves as the landing zone for all incoming healthcare datasets. Its primary objective is to preserve source data exactly as received without applying business transformations or modifying the original records.

Key responsibilities of the Bronze layer include:

- Ingest raw CSV datasets into Delta tables.
- Preserve original schema and source values.
- Maintain traceability to source files.
- Record ingestion statistics within the audit framework.
- Establish a consistent foundation for downstream processing.

Since the Bronze layer represents the system of record, any data quality issues remain intentionally unmodified at this stage and are addressed later within the Silver layer.

3.5 Silver Layer Design

The Silver layer represents the core transformation stage of the pipeline. It converts raw operational data into standardized, validated, and business-ready datasets suitable for analytical processing.

Major responsibilities include:

- Data validation
- Missing value handling
- Duplicate detection
- Standardization of formats
- Business rule implementation
- Business hashing
- Slowly Changing Dimension (SCD Type 2)
- Creation of trusted Silver tables

Unlike the Bronze layer, the Silver layer focuses on improving data quality while preserving business meaning. Each transformation is implemented in a modular manner, allowing future modifications without affecting upstream ingestion.

3.6 Gold Layer Design

The Gold layer provides analytical datasets that support operational reporting and business intelligence.

Instead of storing detailed transactional records, this layer contains curated KPI tables that summarize healthcare operations across multiple business dimensions.

The implemented Gold layer includes:

- Patient KPIs
- Appointment KPIs
- Billing KPIs
- Treatment KPIs
- Daily revenue trends
- Daily appointment trends
- Treatment distribution
- Payment method summaries
- Overall healthcare KPI summary

These datasets are optimized for reporting and can be directly consumed by visualization platforms such as Power BI.

3.7 Metadata-Driven Pipeline Design

To improve maintainability and reduce code duplication, the project adopts a metadata-driven design.

Instead of hardcoding pipeline configuration throughout the notebooks, centralized metadata is used to manage execution parameters and pipeline behavior.

The metadata framework stores information such as:

- Source datasets
- Processing layers

- Table names
- Pipeline configuration
- Execution metadata

This design enables easier maintenance, improves consistency across notebooks, and simplifies future enhancements.

3.8 Modular Notebook Architecture

The implementation divides the pipeline into multiple independent notebooks, each responsible for a single stage of processing.

The modular design provides several benefits:

- Easier debugging
- Better code organization
- Independent testing
- Reduced complexity
- Improved readability
- Reusability of processing logic
- Simplified maintenance

Each notebook performs a clearly defined responsibility and passes standardized outputs to subsequent processing stages.

3.9 Delta Lake Implementation

Delta Lake was selected as the storage layer due to its reliability and support for enterprise data engineering features.

Key capabilities utilized include:

- ACID-compliant transactions
- Schema enforcement
- Schema evolution
- Reliable overwrite operations
- Efficient storage management

These capabilities ensure data consistency across pipeline executions and improve the overall robustness of the platform.

3.10 Audit-Driven Pipeline Design

Operational visibility is an essential requirement for production-grade data pipelines.

Rather than limiting monitoring to notebook execution status, the implementation introduces a centralized audit logging framework that records pipeline activity across every processing stage.

The audit framework captures:

- Batch ID
- Pipeline stage
- Table name
- Processing status
- Records processed
- Execution start time
- Execution end time
- Execution duration
- Source file
- Error messages
- Additional execution remarks

This information enables efficient monitoring, simplifies troubleshooting, and provides a complete execution history for operational analysis.

3.11 Solution Workflow

The complete workflow implemented in this project can be summarized as follows:

Raw Healthcare CSV Files

|



Notebook 00

Environment Setup

Metadata Configuration

Audit Table Creation

|



Notebook 01

Bronze Layer

Raw Data Ingestion

|



Notebook 02

Data Validation

|



Notebook 03

Business Transformation

|



Notebook 04

SCD Type 2

Historical Tracking

|



Notebook 05

Silver Layer Completion

|



Notebook 06

Gold KPIs

Business Analytics



Notebook 07

Audit Reports

Pipeline Monitoring

3.12 Summary

The solution architecture translates the high-level objectives defined in the PDD into a scalable and maintainable healthcare data engineering platform. By combining the Medallion Architecture with Databricks, Apache Spark, Delta Lake, metadata-driven configuration, modular notebooks, and a comprehensive audit framework, the implementation establishes a robust foundation for reliable healthcare analytics.

Beyond satisfying the functional requirements of the PDD, the architecture introduces additional engineering enhancements such as business hashing, standardized metadata management, and improved auditability. These additions strengthen the platform's alignment with enterprise data engineering practices and prepare it for future integration with reporting tools, workflow orchestration, and large-scale healthcare analytics.

Chapter 4 — Security, Privacy & Compliance Engineering

4.1 Introduction

Healthcare data is among the most sensitive categories of organizational information. Patient records often contain personally identifiable information (PII), medical history, appointment details, treatment records, and financial transactions. Unauthorized disclosure or improper handling of such information can compromise patient privacy, violate regulatory requirements, and expose healthcare organizations to significant operational risks.

Although this project was developed using sample healthcare datasets for educational purposes, the solution incorporates security and privacy principles commonly followed in enterprise healthcare data engineering. Throughout the implementation, emphasis was placed on protecting sensitive identifiers, preserving data integrity, maintaining auditability, and designing the pipeline to support future compliance with healthcare regulations such as the Health Insurance Portability and Accountability Act (HIPAA).

4.2 Security Objectives

The security strategy adopted in this project was guided by four primary objectives:

- Protect sensitive healthcare identifiers from unnecessary exposure.
- Maintain data integrity throughout the pipeline.
- Ensure complete traceability of pipeline execution.
- Establish a modular architecture that can support future enterprise security controls.

Rather than treating security as a separate component, these objectives were integrated into the overall pipeline design and implemented across multiple processing stages.

4.3 Healthcare Data Sensitivity

The datasets used in this project represent several core healthcare business entities, including patients, doctors, appointments, treatments, and billing records. Even when using sample data, these entities closely resemble the structure of real-world healthcare information systems.

Examples of sensitive information include:

- Patient identifiers
- Doctor identifiers
- Appointment history

- Treatment records
- Billing information
- Payment details

In production healthcare systems, direct exposure of these identifiers can increase the risk of unauthorized identification and data misuse. Consequently, protecting business identifiers was considered an important design objective during implementation.

4.4 HIPAA Engineering Considerations

The Health Insurance Portability and Accountability Act (HIPAA) establishes standards for protecting sensitive healthcare information within organizations that handle patient data. While this academic project is not intended to be a HIPAA-certified implementation, several engineering decisions were inspired by HIPAA's core principles of privacy, confidentiality, integrity, and accountability.

The implementation incorporates the following HIPAA-aligned considerations:

Confidentiality

Sensitive business identifiers are protected through deterministic hashing before being propagated to downstream analytical layers. This minimizes unnecessary exposure of identifiers while preserving their usefulness for analytical joins and reporting.

Integrity

Data validation rules implemented within the Silver layer help ensure that incomplete, inconsistent, or duplicate records do not compromise analytical outputs.

Accountability

A centralized audit logging framework records execution details for every major pipeline stage, providing traceability and simplifying operational monitoring.

Minimum Necessary Principle

The Gold layer contains aggregated analytical datasets rather than detailed operational records. By exposing summarized business metrics instead of raw transactional information, the solution reduces unnecessary access to sensitive data while still supporting reporting requirements.

Although a production-grade HIPAA implementation would also require identity management, encryption at rest and in transit, access control, and regulatory governance policies, the architectural decisions adopted in this project establish a strong foundation for future compliance.

4.5 Business Hashing Strategy

One of the major enhancements introduced beyond the original PDD is the implementation of business hashing for sensitive healthcare identifiers.

Instead of directly exposing business identifiers throughout the analytical pipeline, deterministic SHA-256 hashing was applied to selected identifier columns during the Silver layer transformation process.

The process can be represented as follows:

Original Business Identifier

|



SHA-256 Hashing

|



Hashed Business Identifier

|



Used in Silver & Gold Layers

This approach ensures that downstream analytical tables no longer rely on raw identifiers while maintaining consistent relationships across datasets.

The use of deterministic hashing enables the same input value to always generate the same hash value. As a result, records originating from different datasets can still be joined accurately without revealing the original business identifiers.

4.6 Why Business Hashing Was Selected

Several approaches were considered for protecting sensitive identifiers during implementation, including encryption, masking, tokenization, and hashing.

Business hashing was selected because it best aligned with the analytical requirements of the project.

The primary reasons include:

- Preserves referential integrity across datasets.

- Produces consistent identifiers for analytical joins.
- Prevents direct exposure of business identifiers.
- Simple to implement within distributed Spark transformations.
- No dependency on external key management infrastructure.
- Minimal computational overhead for batch processing.

Since the objective of the Gold layer is analytical reporting rather than transactional processing, deterministic hashing provides an effective balance between privacy and analytical usability.

4.7 Data Integrity Engineering

Reliable analytics depend on high-quality data. Therefore, several validation mechanisms were incorporated into the Silver layer before business transformations were applied.

These validation processes include:

- Missing value detection
- Duplicate record identification
- Data type validation
- Standardization of business fields
- Removal of inconsistent records
- Schema validation

Performing these checks before downstream transformations improves the reliability of KPI generation and reduces the likelihood of inaccurate business reporting.

4.8 Audit Logging for Accountability

Healthcare data platforms require complete visibility into pipeline execution to support operational monitoring and simplify issue resolution.

To achieve this objective, a centralized audit logging framework was developed as part of the implementation.

The audit framework records:

- Batch ID
- Pipeline stage
- Table name

- Execution status
- Records processed
- Processing start time
- Processing end time
- Execution duration
- Source dataset
- Processing remarks
- Error messages

This information provides complete traceability across every notebook execution and significantly improves operational observability.

Unlike traditional logging mechanisms that only capture notebook execution status, the implemented framework provides business-level monitoring by associating execution metadata with individual pipeline stages and processed datasets.

4.9 Security Decisions Beyond the PDD

While the PDD primarily focused on implementing the Medallion Architecture and data transformations, the final implementation introduces several additional security-oriented enhancements.

Enhancement	Engineering Decision	Benefit
Business Hashing	SHA-256 hashing of sensitive identifiers	Protects business identifiers while preserving analytical relationships
Enhanced Audit Logging	Centralized execution tracking across all notebooks	Improves monitoring, debugging, and accountability
Layered Data Access	Separation of Bronze, Silver, and Gold datasets	Limits unnecessary exposure of raw operational data
Data Validation	Comprehensive validation before business transformations	Improves analytical reliability and data quality
Metadata-Driven Processing	Centralized configuration management	Reduces manual errors and improves consistency

These enhancements extend the original PDD and make the solution more representative of modern enterprise data engineering practices.

4.10 Security Limitations

The implemented security measures are appropriate for the educational scope of this project; however, a production healthcare platform would require additional enterprise-grade security capabilities.

Potential future enhancements include:

- Role-Based Access Control (RBAC)
- Unity Catalog for centralized governance
- Encryption of data at rest and in transit
- Secure secret management using Azure Key Vault or Databricks Secrets
- Multi-factor authentication
- Fine-grained table and column-level access control
- Data lineage and governance
- Automated compliance monitoring
- Row-level and column-level security policies

These features were considered outside the scope of the current implementation but can be integrated without requiring major architectural changes.

4.11 Summary

Security, privacy, and accountability were treated as integral components of the Healthcare Data Platform rather than optional enhancements. Through the implementation of deterministic business hashing, comprehensive audit logging, robust data validation, and a layered Medallion Architecture, the solution minimizes unnecessary exposure of sensitive information while maintaining data integrity and analytical usability.

Although simplified for the purposes of this project, the adopted engineering principles align with industry practices for secure healthcare data platforms and provide a scalable foundation for future enterprise deployments.

Chapter 5 — Notebook-wise Implementation

5.1 Introduction

The Healthcare Data Platform was implemented as a series of modular Databricks notebooks, each responsible for a distinct stage of the Medallion Architecture. This modular approach aligns with the recommendations outlined in the PDD and promotes separation of concerns, easier debugging, code reusability, and simplified maintenance.

Each notebook performs a well-defined task, beginning with environment setup and metadata configuration, followed by data ingestion, validation, transformation, historical tracking, KPI generation, and pipeline monitoring. The output generated by one notebook serves as the input for subsequent stages, creating a structured and scalable end-to-end data engineering workflow.

While the overall implementation follows the design proposed in the PDD, several engineering enhancements were introduced to improve maintainability, auditability, security, and analytical reliability. These enhancements are discussed individually within the corresponding notebook sections.

For consistency, each notebook is documented using the following structure:

- PDD Requirement
- Objective
- Implementation Overview
- Workflow
- Functions Developed
- Design Decisions
- Business Rules
- Changes from the PDD
- Benefits of the Changes
- Outputs Generated

This standardized approach provides a comprehensive understanding of both the implementation and the engineering rationale behind each stage of the pipeline.

5.2 Notebook 00 – Environment Setup & Metadata Configuration

5.2.1 Overview

Notebook 00 serves as the foundation of the Healthcare Data Platform. Before any healthcare datasets can be ingested or transformed, the processing environment must be configured and the metadata required for pipeline execution must be established.

This notebook initializes the project by creating the centralized metadata configuration and audit logging framework that are shared across all subsequent notebooks. Rather than embedding configuration values directly within individual notebooks, the implementation follows a metadata-driven approach that improves consistency, reduces redundancy, and simplifies future maintenance.

By establishing these foundational components at the beginning of the pipeline, all downstream notebooks are able to execute using standardized configuration and centralized monitoring.

5.2.2 PDD Requirement

According to the Project Definition Document (PDD), the initial notebook is responsible for preparing the execution environment required by the healthcare data pipeline. This includes creating the metadata structures needed to manage pipeline execution and establishing an audit mechanism capable of tracking the progress and status of each processing stage.

The PDD emphasizes the importance of centralized configuration and operational monitoring to ensure that the pipeline remains scalable, maintainable, and easy to troubleshoot.

5.2.3 Objective

The primary objectives of Notebook 00 are:

- Configure the project environment.
 - Create centralized metadata required for pipeline execution.
 - Initialize the audit logging framework.
 - Standardize configuration used across all notebooks.
 - Establish the foundation for a metadata-driven pipeline.
-

5.2.4 Implementation Overview

The notebook begins by defining the project configuration required throughout the pipeline. Instead of storing configuration values separately within each notebook, a centralized metadata configuration is created to ensure consistency across all processing stages.

Following the metadata setup, the notebook creates the audit log table that records execution information for every pipeline stage. The audit framework captures processing metadata such as execution status, processed record counts, timestamps, source files, and execution remarks.

These two components—the metadata configuration and audit log—serve as shared infrastructure that supports every subsequent notebook in the project.

5.2.5 Workflow

The execution workflow of Notebook 00 is summarized below:

Initialize Project

|



Create Metadata Configuration

|



Create Audit Log Table

|



Validate Configuration

|



Environment Ready

5.2.6 Metadata Configuration

A metadata-driven pipeline was adopted to improve maintainability and eliminate the need for hardcoded configuration values across multiple notebooks. Instead of repeatedly defining dataset paths, table names, and processing parameters, a centralized metadata configuration was created during the environment setup phase.

The metadata configuration serves as a single source of truth for the entire pipeline. Subsequent notebooks reference this configuration to determine the appropriate datasets, processing stages, and destination tables, ensuring consistent behavior throughout the implementation.

This design significantly reduces code duplication and simplifies future modifications, as configuration changes can be made centrally without requiring updates to multiple notebooks.

5.2.7 Audit Logging Framework

To provide operational visibility and simplify pipeline monitoring, an audit logging framework was established during the initialization phase.

The audit log captures execution details for every major processing stage, allowing developers and administrators to monitor pipeline performance, identify failures, and analyze execution history without manually inspecting notebook outputs.

The implemented audit table contains the following fields:

Column	Description
batch_id	Unique identifier for each pipeline execution
pipeline_stage	Processing stage being executed
table_name	Delta table associated with the execution
status	Execution status (Success / Failed)
records_processed	Number of records processed
start_time	Pipeline execution start time
end_time	Pipeline execution completion time
remarks	Additional execution information
source_file	Source dataset processed
error_message	Error details in case of execution failure

This centralized audit framework remains active throughout the pipeline lifecycle and is updated by every subsequent notebook.

5.2.8 Functions Developed

Notebook 00 primarily focuses on project initialization and therefore contains only foundational utility functions.

Function	Purpose
<code>create_metadata_config()</code> <i>(or your actual implementation)</i>	Creates the centralized metadata configuration used throughout the pipeline.
<code>create_audit_table()</code> <i>(or your actual implementation)</i>	Creates the audit log table for pipeline monitoring.
<code>log_audit()</code> <i>(if defined here)</i>	Records execution details into the audit log.

Note: Replace the function names above with the exact names used in your notebook.

5.2.9 Engineering Decisions

Several architectural decisions were made during the implementation of Notebook 00 to improve the overall quality and maintainability of the pipeline.

Decision 1 – Metadata-Driven Configuration

Decision

A centralized metadata configuration was implemented instead of storing configuration values independently within each notebook.

Reason

Hardcoded values increase maintenance effort and make future modifications error-prone.

Benefit

- Improved maintainability
 - Centralized configuration management
 - Easier scalability for additional datasets
-

Decision 2 – Centralized Audit Framework

Decision

A reusable audit logging framework was implemented before data processing begins.

Reason

Pipeline execution should be traceable across every processing stage.

Benefit

- Simplified debugging

- Better operational monitoring
 - Consistent execution tracking
-

Decision 3 – Modular Initialization

Decision

Environment initialization was isolated into a dedicated notebook.

Reason

Separating setup logic from transformation logic improves modularity and keeps subsequent notebooks focused on business processing.

Benefit

- Better code organization
 - Easier testing
 - Improved readability
-

5.2.10 Changes Compared to the PDD

Although the PDD proposed an environment setup notebook, the implementation introduces several enhancements to improve scalability and maintainability.

PDD Expectation	Final Implementation	Reason	Benefit
Basic environment setup	Metadata-driven configuration	Centralized management	Easier maintenance
Audit table creation	Enhanced audit schema with execution metadata	Better observability	Improved monitoring and debugging
Notebook initialization	Reusable initialization framework	Reduce redundancy	Consistent execution across notebooks

5.2.11 Benefits Achieved

The implementation of Notebook 00 establishes the foundation upon which the remaining pipeline is built. By centralizing configuration and introducing a reusable audit framework, the project achieves several long-term benefits:

- Consistent configuration across all notebooks.
 - Reduced code duplication through centralized metadata management.
 - Improved pipeline observability via structured audit logging.
 - Simplified maintenance and future enhancements.
 - Strong foundation for scalable and modular data engineering workflows.
-

5.2.12 Outputs Generated

Upon successful execution, Notebook 00 produces the following foundational components:

Output	Purpose
Metadata Configuration Stores	centralized pipeline configuration for downstream notebooks.
Audit Log Table	Records execution details for all pipeline stages.
Initialized Environment	Prepares the workspace for Bronze layer data ingestion.

5.3 Notebook 01 – Bronze Layer Data Ingestion

5.3.1 Purpose

The objective of this notebook is to ingest raw healthcare datasets into the Bronze layer while preserving the source data in its original form. No validation or business transformations are performed at this stage, ensuring the Bronze layer serves as a reliable source of truth for downstream processing.

5.3.2 PDD Requirement

The PDD specifies that the Bronze layer should:

- Ingest raw healthcare datasets.
 - Preserve original records without modification.
 - Store data in Delta tables.
 - Record execution details through the audit framework.
-

5.3.3 Implementation

The notebook reads five healthcare datasets—Patients, Doctors, Appointments, Treatments, and Billing—from the source location and stores them as Delta tables in the Bronze layer. During ingestion, execution details such as processed records, execution status, timestamps, and source information are captured using the centralized audit framework created in Notebook 00.

5.3.4 Functions Developed

Function	Purpose
load_to_bronze() <i>(replace with actual name)</i>	Loads a dataset into the corresponding Bronze table.
log_audit()	Logs execution details after ingestion.

5.3.5 Engineering Decisions

Decision	Reason	Benefit
Preserve raw data without modification	Maintain source integrity	Supports reprocessing and traceability
Store data in Delta format	Reliable and efficient storage	ACID transactions and better performance
Integrate audit logging	Monitor every ingestion step	Easier debugging and operational visibility

5.3.6 Changes from the PDD

PDD	Implementation	Benefit
Basic ingestion	Standardized ingestion workflow with reusable functions	Reduced code duplication
Bronze table creation	Integrated audit logging	Better monitoring and debugging

5.3.7 Outputs

Bronze Tables Created

- bronze_patients
- bronze_doctors
- bronze_appointments
- bronze_treatments
- bronze_billing

Additionally, audit entries are generated for each ingestion process.

5.4 Notebook 02 – Data Validation & Cleansing

5.4.1 Purpose

The objective of this notebook is to improve the quality of the ingested healthcare data before business transformations are applied. It performs validation checks to identify missing values, duplicate records, invalid data, and schema inconsistencies, ensuring that only reliable data progresses to the next stage of the pipeline.

5.4.2 PDD Requirement

According to the PDD, the Silver layer should validate and cleanse the raw healthcare datasets to improve data quality and consistency before analytical processing. The notebook is expected to implement validation rules, handle data quality issues, and prepare standardized datasets for business transformations.

5.4.3 Implementation

The notebook reads the Bronze Delta tables and applies a series of validation checks on each dataset. These checks include identifying null values, removing duplicate records, validating data types, and ensuring mandatory fields are populated. Invalid or inconsistent records are handled according to predefined business rules before the cleaned datasets are passed to the transformation stage.

Throughout the validation process, execution details are recorded in the centralized audit log to maintain complete pipeline traceability.

5.4.4 Validation Rules Implemented

The following validation checks were applied across the healthcare datasets:

- Detection of missing or null values in mandatory columns.

- Removal of duplicate records.
- Validation of data types and schema consistency.
- Standardization of date and categorical fields.
- Verification of mandatory business attributes before further processing.

5.4.5 Engineering Decisions

Decision	Reason	Benefit
Validate data before transformation	Prevent propagation of poor-quality data	Improves analytical accuracy
Apply reusable validation logic	Reduce repetitive code	Better maintainability
Integrate audit logging	Track validation results	Simplifies monitoring and debugging

5.4.6 Changes from the PDD

PDD	Implementation	Benefit
Data validation	Modular validation workflow	Easier maintenance
Basic quality checks	Centralized audit integration	Better pipeline observability

5.4.7 Outputs

The notebook produces validated healthcare datasets that are ready for business transformation in the subsequent processing stage. Audit records are also generated to capture validation statistics and execution status.

5.5 Notebook 03 – Business Transformations

5.5.1 Purpose

The objective of this notebook is to transform the validated healthcare data into a standardized format suitable for analytical processing. Business rules are applied to enrich, standardize, and prepare the data before loading it into the Silver layer.

5.5.2 PDD Requirement

The PDD specifies that the Silver layer should implement business transformations to convert validated raw data into structured datasets that support reporting and downstream analytics.

5.5.3 Implementation

This notebook reads the validated Bronze datasets and applies business-specific transformation rules. The transformations include standardizing data formats, deriving required attributes, formatting date columns, and preparing datasets according to the business requirements defined in the PDD.

The transformed datasets are then written to the Silver layer as Delta tables. Audit information, including execution status and processed record counts, is logged after successful completion.

5.5.4 Major Transformations

The following business transformations were implemented:

- Standardization of date and timestamp formats.
 - Formatting of categorical values for consistency.
 - Derivation of business attributes required for reporting.
 - Data normalization to maintain a consistent schema across datasets.
 - Preparation of clean datasets for historical tracking and KPI generation.
-

5.5.5 Engineering Decisions

Decision	Reason	Benefit
Separate validation and transformation logic	Better modularity	Easier maintenance
Standardize business data	Ensure consistency	Reliable analytics
Store transformed data in Silver layer	Support downstream processing	Improved query performance

5.5.6 Changes from the PDD

PDD	Implementation	Benefit
Business transformations	Modular transformation pipeline	Better code organization
Standardized datasets	Reusable transformation logic	Improved scalability

5.5.7 Outputs

The notebook generates the transformed Silver datasets:

- silver_patients
- silver_appointments
- silver_billing
- silver_treatments

These datasets serve as the foundation for historical tracking and KPI generation.

5.6 Notebook 04 – Historical Data Management (SCD Type 2 & Business Hashing)

5.6.1 Purpose

The objective of this notebook is to maintain historical versions of healthcare records while ensuring accurate change detection. It implements Slowly Changing Dimension (SCD) Type 2 to preserve historical data and uses business hashing to efficiently identify changes between incoming and existing records.

5.6.2 PDD Requirement

The PDD requires the Silver layer to support historical data management by tracking changes in business entities rather than overwriting existing records. It also recommends implementing a mechanism to efficiently detect record updates for incremental processing.

5.6.3 Implementation

The notebook processes the transformed Silver datasets and compares incoming records with existing data using deterministic business hashes. If no changes are detected, the existing record remains unchanged. When differences are identified, the current record is marked as inactive, and a new version is inserted with updated values and validity dates.

This approach ensures that the complete history of patient and transactional data is preserved while supporting efficient incremental updates.

5.6.4 SCD Type 2 Implementation

The SCD Type 2 process follows the workflow below:

Incoming Record

|



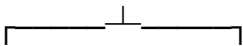
Generate Business Hash

|



Compare with Existing Record

|



| |

No Change Change Detected

| |



Skip Expire Current Record

|



Insert New Version

Key features of the implementation include:

- Existing records are never overwritten.
- Historical versions are preserved.
- Active and inactive records are maintained using status flags.
- Effective start and end dates track record validity.

5.6.5 Business Hashing

To avoid comparing every column individually, a deterministic SHA-256 business hash is generated using selected business attributes. During each pipeline execution, the newly generated hash is compared with the stored hash to determine whether a record has changed. This approach significantly improves change detection while reducing processing complexity and ensuring consistent results across executions.

5.6.6 Engineering Decisions

Decision	Reason	Benefit
Implement SCD Type 2	Preserve historical records	Complete data history
Use SHA-256 business hashing	Efficient change detection	Faster incremental processing
Maintain active/inactive records	Support historical reporting	Improved analytical accuracy

5.6.7 Changes from the PDD

PDD	Final Implementation	Benefit
Historical tracking	SCD Type 2 with validity periods	Complete record history
Change detection	SHA-256 business hashing	Efficient incremental updates
Incremental processing	Automated comparison workflow	Improved scalability

5.6.8 Outputs

The notebook updates the Silver tables by maintaining historical versions of records while ensuring that only the latest version remains active. These datasets become the primary source for Gold layer KPI generation.

5.7 Notebook 05 – Silver Layer Finalization

5.7.1 Purpose

The objective of this notebook is to consolidate the validated and transformed healthcare datasets into the final Silver layer. It ensures that the processed data is standardized, consistent, and ready for analytical reporting in the Gold layer.

5.7.2 PDD Requirement

According to the PDD, the Silver layer should provide clean, structured, and business-ready datasets that serve as the foundation for KPI generation and reporting.

5.7.3 Implementation

This notebook stores the processed datasets as final Silver Delta tables after completing validation, business transformations, and historical data management. It verifies data consistency, updates audit records, and prepares the datasets for Gold layer processing.

The final Silver layer contains reliable and analytics-ready data while preserving historical records generated through the SCD Type 2 process.

5.7.4 Final Silver Tables

The following business-ready datasets are created:

- silver_patients
- silver_appointments
- silver_billing
- silver_treatments

These tables act as the trusted source for KPI generation and dashboard reporting.

5.7.5 Engineering Decisions

Decision	Reason	Benefit
Store processed data in Silver layer	Separate processing from analytics	Improved modularity
Use Delta tables	Reliable storage and querying	Better performance
Update audit logs	Track pipeline completion	Enhanced monitoring

5.7.6 Changes from the PDD

PDD	Final Implementation	Benefit
Clean datasets	Standardized Silver tables	Consistent analytics

PDD	Final Implementation	Benefit
Pipeline tracking	Audit integration	Better operational visibility

5.7.7 Outputs

The notebook produces the finalized Silver datasets, which are used as the input for Gold layer KPI generation.

5.8 Notebook 06 – Gold Layer KPI Generation

5.8.1 Purpose

The objective of this notebook is to transform the curated Silver datasets into business-ready Key Performance Indicators (KPIs). These Gold layer tables provide aggregated insights that support healthcare reporting, operational monitoring, and future dashboard development.

5.8.2 PDD Requirement

The PDD specifies that the Gold layer should generate analytical datasets by aggregating processed healthcare data into meaningful business metrics for reporting and decision-making.

5.8.3 Implementation

This notebook reads the finalized Silver tables and performs aggregations to generate healthcare KPIs. Business metrics related to patients, appointments, billing, treatments, revenue, and payment methods are calculated and stored as separate Gold Delta tables.

Unlike the Silver layer, which focuses on data quality and consistency, the Gold layer is optimized for analytical queries and reporting. Audit logs are updated after the successful creation of each KPI table.

5.8.4 KPI Tables Generated

The following analytical tables are created:

Gold Table	Purpose
gold_patient_kpi	Patient-related metrics
gold_appointment_kpi	Appointment statistics

Gold Table	Purpose
gold_billing_kpi	Billing and revenue analysis
gold_treatment_kpi	Treatment summary
gold_kpi_summary	Overall healthcare KPIs
gold_daily_revenue_trend	Daily revenue trends
gold_daily_appointment_trend	Daily appointment trends
gold_treatment_distribution	Treatment distribution analysis
gold_payment_method_summary	Payment method statistics

5.8.5 Engineering Decisions

Decision	Reason	Benefit
Separate KPIs into multiple tables	Improve modularity	Easier reporting and maintenance
Store KPIs in Gold layer	Optimize analytical queries	Faster reporting
Generate trend-based tables	Support future dashboards	Better business insights

5.8.6 Changes from the PDD

PDD	Final Implementation	Benefit
KPI generation	Multiple specialized KPI tables	More detailed analytics
Business reports	Trend and summary tables	Improved decision support

5.8.7 Outputs

The notebook produces nine Gold Delta tables containing aggregated healthcare metrics. These tables represent the final analytical output of the data pipeline and are designed to support business reporting and future Power BI dashboard integration.

5.9 Notebook 07 – Audit Report & Pipeline Monitoring

5.9.1 Purpose

The objective of this notebook is to provide visibility into pipeline execution by analyzing the audit logs generated throughout the ETL process. It enables monitoring of pipeline performance, execution status, processing statistics, and failures, supporting easier debugging and operational management.

5.9.2 PDD Requirement

The PDD recommends implementing an audit and monitoring mechanism to track pipeline execution, record processing statistics, and simplify troubleshooting through centralized logging.

5.9.3 Implementation

This notebook queries the centralized audit log created in Notebook 00 and generates reports summarizing pipeline execution. It provides insights into the latest pipeline run, execution status, records processed, execution time, and any failures encountered during processing. These reports help administrators verify successful execution and quickly identify issues without manually reviewing each notebook.

5.9.4 Audit Reports Generated

The notebook generates the following reports:

Report	Purpose
Latest Execution Report	Displays details of the most recent pipeline run
Pipeline Status Report	Shows the execution status of each stage
Execution Time Analysis	Measures processing duration for each notebook
Failure Analysis Report	Identifies failed stages and error messages
Records Processed Report	Summarizes the number of records processed
Pipeline Summary Report	Provides an overall execution summary

5.9.5 Engineering Decisions

Decision	Reason	Benefit
Centralized audit reporting	Single source for pipeline monitoring	Easier troubleshooting
Track execution metrics	Measure pipeline performance	Better operational insights
Record failures with error messages	Simplify issue diagnosis	Faster debugging

5.9.6 Changes from the PDD

PDD	Final Implementation	Benefit
Audit logging	Comprehensive audit reporting	Improved pipeline monitoring
Basic execution tracking	Multiple analytical audit reports	Better operational visibility

5.9.7 Outputs

The notebook generates multiple audit reports that summarize the execution of the complete Healthcare Data Platform. These reports provide insights into pipeline health, execution performance, processed records, and failures, supporting efficient monitoring and maintenance.

Chapter 6 – Engineering Decisions & Design Rationale

6.1 Introduction

Developing a scalable healthcare data platform requires more than simply implementing an ETL pipeline. It involves selecting appropriate architectural patterns, storage technologies, and processing strategies that ensure maintainability, scalability, reliability, and data quality.

This chapter discusses the key engineering decisions made during the implementation of the Healthcare Data Platform. Each decision was taken to address specific business and technical requirements while improving the overall efficiency and robustness of the solution.

6.2 Adoption of Medallion Architecture

The Medallion Architecture was selected as the foundation of the pipeline because it separates data processing into distinct stages: Bronze, Silver, and Gold.

This layered approach improves data quality, simplifies debugging, and allows each processing stage to focus on a specific responsibility. Raw data is preserved in the Bronze layer, validated and transformed in the Silver layer, and finally aggregated into business-ready KPIs in the Gold layer.

Benefits

- Modular pipeline design
 - Improved maintainability
 - Better data quality management
 - Simplified troubleshooting
-

6.3 Use of Delta Lake

Delta Lake was chosen as the storage format for all Bronze, Silver, and Gold tables due to its reliability and performance advantages over traditional file-based storage.

Its support for ACID transactions, schema enforcement, and efficient query execution makes it well suited for enterprise-scale data engineering pipelines.

Benefits

- Reliable data storage
- Faster analytical queries

- Improved data consistency
 - Scalable architecture
-

6.4 Metadata-Driven Pipeline

Instead of hardcoding configuration values across notebooks, a centralized metadata configuration was implemented.

This approach enables all notebooks to use a common configuration source, making the pipeline easier to maintain and extend as new datasets or processing stages are introduced.

Benefits

- Reduced code duplication
 - Easier maintenance
 - Centralized configuration management
 - Better scalability
-

6.5 Historical Data Management using SCD Type 2

Healthcare data frequently changes over time, making historical tracking essential. To preserve previous versions of records, the pipeline implements Slowly Changing Dimension (SCD) Type 2.

Rather than overwriting existing records, new versions are inserted while older versions are marked inactive. This enables historical reporting and maintains a complete record lifecycle.

Benefits

- Preserves historical information
 - Supports audit requirements
 - Enables time-based analysis
 - Prevents data loss
-

6.6 Business Hashing for Change Detection

To efficiently identify modified records, deterministic SHA-256 business hashes were generated using selected business attributes.

Instead of comparing multiple columns individually, hash values are compared to determine whether a record has changed, improving the efficiency of incremental processing.

Benefits

- Faster change detection
 - Reduced comparison complexity
 - Consistent incremental updates
 - Improved scalability
-

6.7 Centralized Audit Logging

A centralized audit framework was implemented to monitor every stage of the pipeline. Execution status, processed records, timestamps, and error information are recorded in a common audit table.

This provides complete visibility into pipeline execution and simplifies troubleshooting.

Benefits

- End-to-end pipeline monitoring
 - Faster debugging
 - Execution traceability
 - Improved operational visibility
-

6.8 Modular Notebook Design

The pipeline was divided into independent notebooks, each responsible for a specific processing stage.

This modular design simplifies development, testing, debugging, and future enhancements while allowing individual pipeline stages to be modified without impacting the overall workflow.

Benefits

- Improved readability
 - Easier maintenance
 - Better code reusability
 - Independent testing of pipeline stages
-

6.9 Chapter Summary

The engineering decisions described in this chapter contributed to the development of a scalable, maintainable, and reliable healthcare data platform. By combining the Medallion Architecture, Delta Lake, metadata-driven configuration, SCD Type 2, business hashing, centralized audit logging, and modular notebook design, the implemented solution extends beyond the baseline requirements of the PDD and provides a strong foundation for future enhancements and enterprise-scale deployment.

Chapter 7 – PDD vs Final Implementation

7.1 Introduction

The Project Definition Document (PDD) served as the blueprint for designing and implementing the Healthcare Data Platform. While the core architecture and functional requirements were retained, several implementation decisions were made to improve scalability, maintainability, monitoring, and data quality.

This chapter compares the original PDD requirements with the final implementation and highlights the enhancements introduced during development.

7.2 PDD vs Final Implementation

PDD Requirement	Final Implementation	Improvement
Medallion Architecture	Implemented Bronze, Silver, and Gold layers	Fully aligned with PDD
Raw data ingestion	Bronze Delta tables with audit logging	Enhanced monitoring
Data validation	Modular validation and cleansing	Improved data quality
Business transformations	Standardized transformation pipeline	Better maintainability
Historical data management	SCD Type 2 with business hashing	Efficient change tracking
KPI generation	Nine Gold KPI tables	More comprehensive analytics
Audit logging	Centralized audit framework with reports	Improved operational visibility
Dashboard support	KPI tables prepared for Power BI integration	Ready for future BI reporting

7.3 Key Enhancements

The final implementation introduced several enhancements beyond the baseline requirements defined in the PDD:

- Centralized metadata-driven configuration.

- Deterministic SHA-256 business hashing for efficient change detection.
- Comprehensive audit logging and execution reporting.
- Modular notebook design for easier maintenance and scalability.
- Additional KPI tables supporting trend analysis and future dashboard integration.

These enhancements improve the usability and maintainability of the platform while preserving alignment with the original project objectives.

7.4 Chapter Summary

The implemented Healthcare Data Platform successfully satisfies the functional requirements specified in the PDD while incorporating additional engineering improvements that enhance scalability, data quality, and operational monitoring. These enhancements strengthen the overall solution and provide a solid foundation for future extensions.

Chapter 8 – Results, Challenges & Future Enhancements

8.1 Project Results

The Healthcare Data Platform was successfully implemented using Databricks and the Medallion Architecture, meeting the functional requirements defined in the Project Definition Document (PDD). The solution provides an end-to-end data engineering pipeline capable of ingesting, validating, transforming, and analyzing healthcare data while maintaining historical records and comprehensive audit logs.

The final implementation delivers a modular, scalable, and maintainable architecture that supports reliable analytics and future business intelligence integration.

8.2 Key Deliverables

The following deliverables were successfully completed during the project:

Deliverable	Status
Metadata-driven pipeline	✓ Completed
Bronze layer data ingestion	✓ Completed
Silver layer validation and transformation	✓ Completed
SCD Type 2 historical tracking	✓ Completed
Business hashing for change detection	✓ Completed
Gold layer KPI generation	✓ Completed
Centralized audit logging	✓ Completed
Pipeline monitoring reports	✓ Completed

8.3 Challenges Faced and Solutions

During the implementation, several technical challenges were encountered. These were addressed through appropriate engineering decisions to improve the reliability and maintainability of the pipeline.

Challenge	Solution Implemented
Maintaining historical records	Implemented SCD Type 2 to preserve record history.

Challenge	Solution Implemented
Efficient change detection	Used SHA-256 business hashing instead of comparing multiple columns.
Ensuring data quality	Applied validation and cleansing before transformations.
Monitoring pipeline execution	Developed a centralized audit logging framework.
Managing configuration across notebooks	Adopted a metadata-driven configuration approach.

8.4 Future Enhancements

Although the implemented solution meets the project objectives, several enhancements can further improve its capabilities:

- Integrate **Power BI** for interactive dashboards and reporting.
 - Support **real-time data ingestion** using Structured Streaming.
 - Implement **Unity Catalog** for centralized data governance.
 - Automate deployment using **CI/CD pipelines**.
 - Introduce **Role-Based Access Control (RBAC)** for enhanced security.
 - Deploy the solution on a cloud platform for improved scalability and availability.
-

8.5 Chapter Summary

The project successfully delivers a complete healthcare data engineering solution with robust data processing, historical tracking, KPI generation, and audit capabilities. The challenges encountered during development were effectively addressed through sound engineering practices, while the proposed future enhancements provide a clear roadmap for evolving the platform into an enterprise-ready solution.

Chapter 9 – Conclusion

9.1 Conclusion

This project presented the design and implementation of a Healthcare Data Platform using Databricks and the Medallion Architecture. The solution successfully transformed raw healthcare datasets into reliable analytical outputs through a structured pipeline consisting of data ingestion, validation, business transformation, historical data management, KPI generation, and audit reporting.

The implementation fulfilled the objectives defined in the Project Definition Document while incorporating additional engineering enhancements such as metadata-driven configuration, SCD Type 2, business hashing, and centralized audit logging. These improvements increased the scalability, maintainability, and reliability of the overall solution.

Beyond meeting the project requirements, this work demonstrates the practical application of modern data engineering principles in building a secure, modular, and analytics-ready healthcare platform. The proposed future enhancements, including Power BI integration, real-time processing, and cloud deployment, provide opportunities to further extend the solution for enterprise-scale healthcare analytics.

Chapter 10 – References & Appendix

10.1 References

The implementation and design of the Healthcare Data Platform were based on industry-standard documentation and best practices. The following resources were referenced during the development of the project.

Books

1. Kimball, R., & Ross, M. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling* (3rd Edition). Wiley.
2. Inmon, W. H. *Building the Data Warehouse* (4th Edition). Wiley.

Documentation

3. Databricks Documentation – <https://docs.databricks.com/>
4. Delta Lake Documentation – <https://docs.delta.io/>
5. Apache Spark Documentation – <https://spark.apache.org/docs/latest/>
6. Microsoft Learn – <https://learn.microsoft.com/>
7. HIPAA Privacy Rule Summary – <https://www.hhs.gov/hipaa/index.html>

Project Documents

8. Healthcare Data Platform – Project Definition Document (PDD)
9. Project notebooks and implementation artifacts developed during the project.

10.2 Appendix

Appendix A – Notebook List

Notebook	Description
Notebook 00	Environment Setup & Metadata Configuration
Notebook 01	Bronze Layer Data Ingestion
Notebook 02	Data Validation & Cleansing
Notebook 03	Business Transformations
Notebook 04	Historical Data Management (SCD Type 2 & Business Hashing)
Notebook 05	Silver Layer Finalization

Notebook	Description
Notebook 06	Gold Layer KPI Generation
Notebook 07	Audit Report & Pipeline Monitoring

Appendix B – Gold Layer Output Tables

Table	Description
gold_patient_kpi	Patient-related KPIs
gold_appointment_kpi	Appointment statistics
gold_billing_kpi	Billing and revenue metrics
gold_treatment_kpi	Treatment-related KPIs
gold_kpi_summary	Overall healthcare summary
gold_daily_revenue_trend	Daily revenue analysis
gold_daily_appointment_trend	Daily appointment trends
gold_treatment_distribution	Treatment distribution analysis
gold_payment_method_summary	Payment method statistics

Appendix C – Audit Log Schema

Column	Description
batch_id	Unique pipeline execution identifier
pipeline_stage	Processing stage
table_name	Table processed
status	Execution status
records_processed	Number of records processed
start_time	Execution start time
end_time	Execution end time

Column	Description
remarks	Additional execution information
source_file	Source dataset
error_message	Error details (if any)

Appendix D – Project Architecture Diagrams

Include the following diagrams:

- Overall Solution Architecture
 - Medallion Architecture
 - Pipeline Workflow
 - Data Flow Diagram
 - SCD Type 2 Workflow
 - Business Hashing Process
 - Folder Structure
 - KPI Generation Workflow
-

Appendix E – Additional Screenshots

Include screenshots that support the implementation but are not discussed in the main report, such as:

- Databricks Workspace
- Unity Catalog tables
- Delta table previews
- Audit reports
- Gold KPI outputs
- Notebook execution summaries